

lab5 简化路由器程序设计

才子桓 2212700 密码科学与技术

实验要求

简单路由器程序设计实验的具体要求为：

1. 设计和实现一个路由器程序，要求完成的路由器程序能和现有的路由器产品（如思科路由器、华为路由器、微软的路由器等）进行协同工作。
2. 程序可以仅实现IP数据报的获取、选路、投递等路由器要求的基本功能。可以忽略分片处理、选项处理、动态路由表生成等功能。
3. 需要给出路由表的建立和形成方式。
4. 需要给出路由器的工作日志，显示数据报获取和转发过程。
5. 完成的程序须通过现场测试，并讲解和报告自己的设计思路、开发和实现过程、测试方法和过程。

课堂检查：

需要大家以个人形式制作一份演示文档，现场讲解一下实验的内容，并最后在系统中提交。

要求：

1. 页面简洁有序，避免文字堆砌，建议多图少字
2. 按照要求，需要有设计思路、开发和实现过程、测试方法和过程的内容，有关测试的内容可以当堂完成。
3. 着重以模块形式讲解你的代码设计

预期结果：

预期实验结果

1. 获取并显示本机单网卡的双IP地址和MAC地址;
2. 可以显示路由表，可以添加路由表项(206.1.3.0/255.255.255.0/206.1.2.2)，可以删除路由表项但无法删除默认路由表项;
3. 显示工作日志，输出数据报获取和转发过程(至少展示数据报的源IP、目的IP、下一跳信息);
4. 主机A和B可以互相ping通，且显示的TTL正确(126或62);

代码涉及的内容

1. 打开网卡获取双IP;
2. 伪造ARP报文获取本机MAC;
3. 自动添加默认路由表项，手动添加&删除路由表项，显示路由表;
4. 捕获报文的过滤条件（ARP & IP）；
5. 捕获IP报文的处理;
6. 捕获ARP报文的处理;
7. 缓冲区的超时删除;

附件：（在文件夹中打开）

[lab5 演示文档](#)

[lab5 源码](#)

实验环境

操作机：Windows Server 2003

编译机：Windows 11

编译语言：C++ 14

编译IDE：Visual Studio 2022

环境配置

由于虚拟环境 Windows Server 2003 版本较老，所以编译过程需要先将 Visual Studio 2022 进行设置：

1. 项目->属性->链接器->系统->所需最低版本，填入 5.01
2. 编译选项设置为 Release 和 x86



3 号虚拟机（路由器 2）的配置：

1. 打开 cmd，添加路由表项：

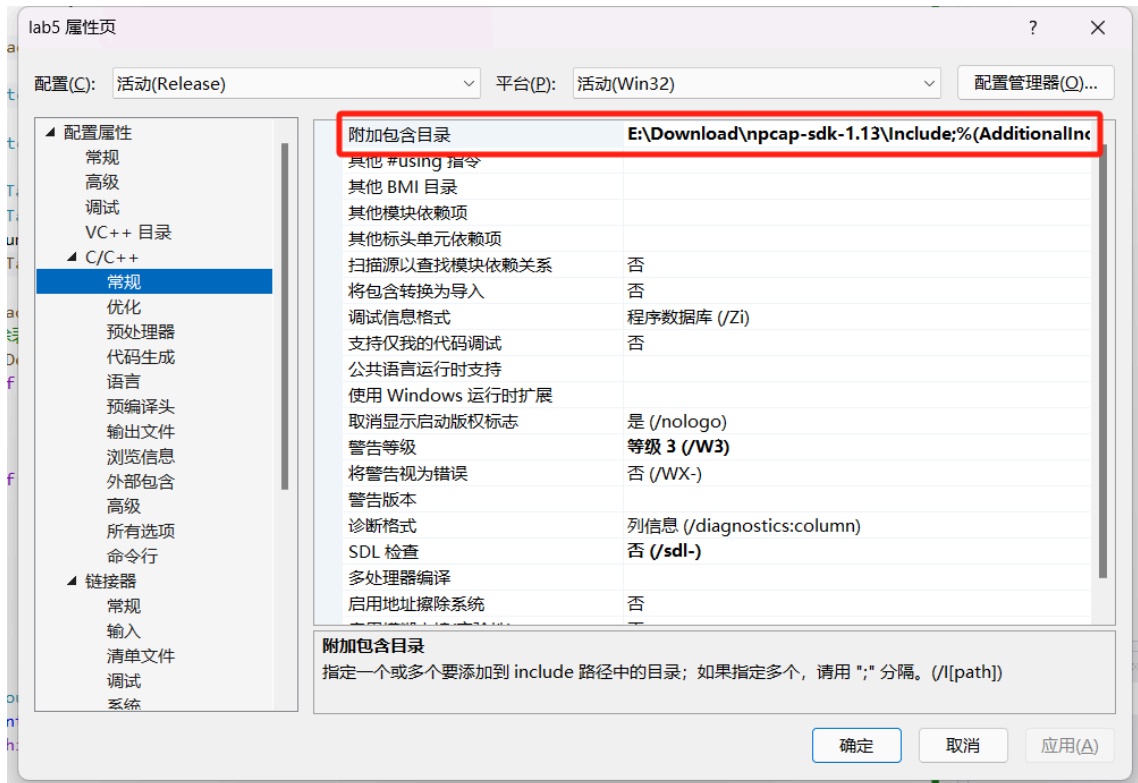
```
route ADD 206.1.1.0 MASK 255.255.255.0 206.1.2.1
```

2. 验证是否添加成功：

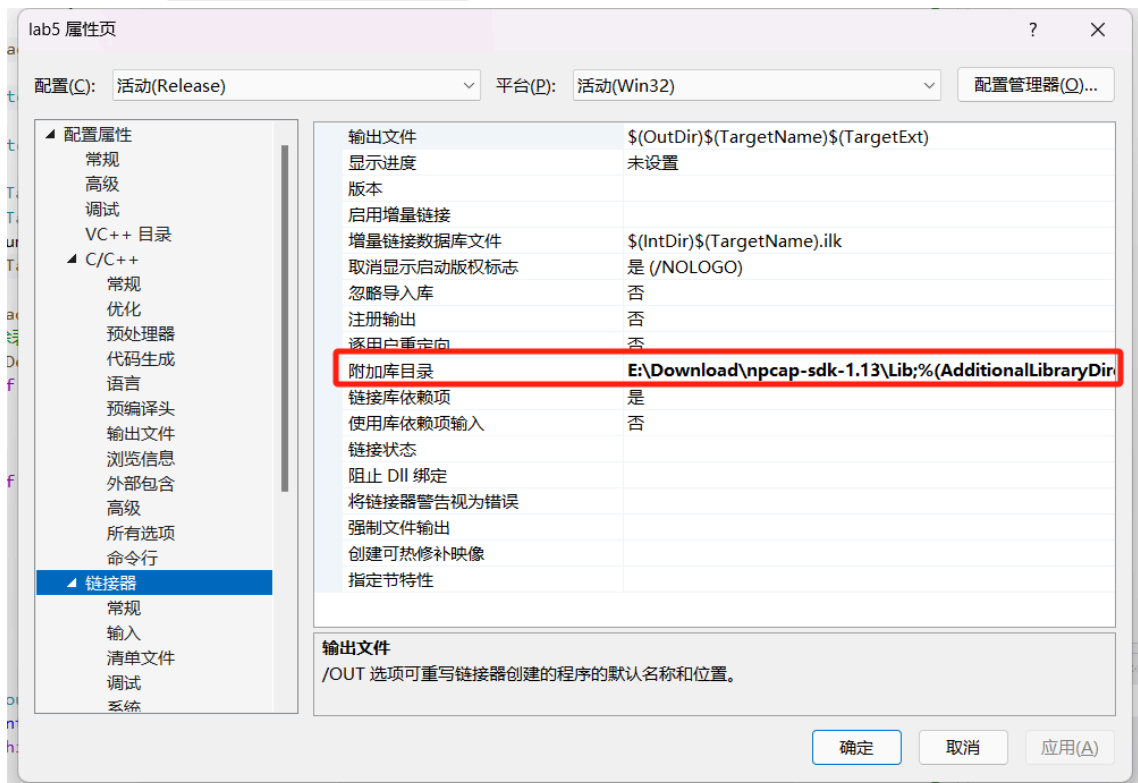
```
route print
```

npcap 配置：

1. 附加包含目录: ..\npcap-sdk-1.13\Include

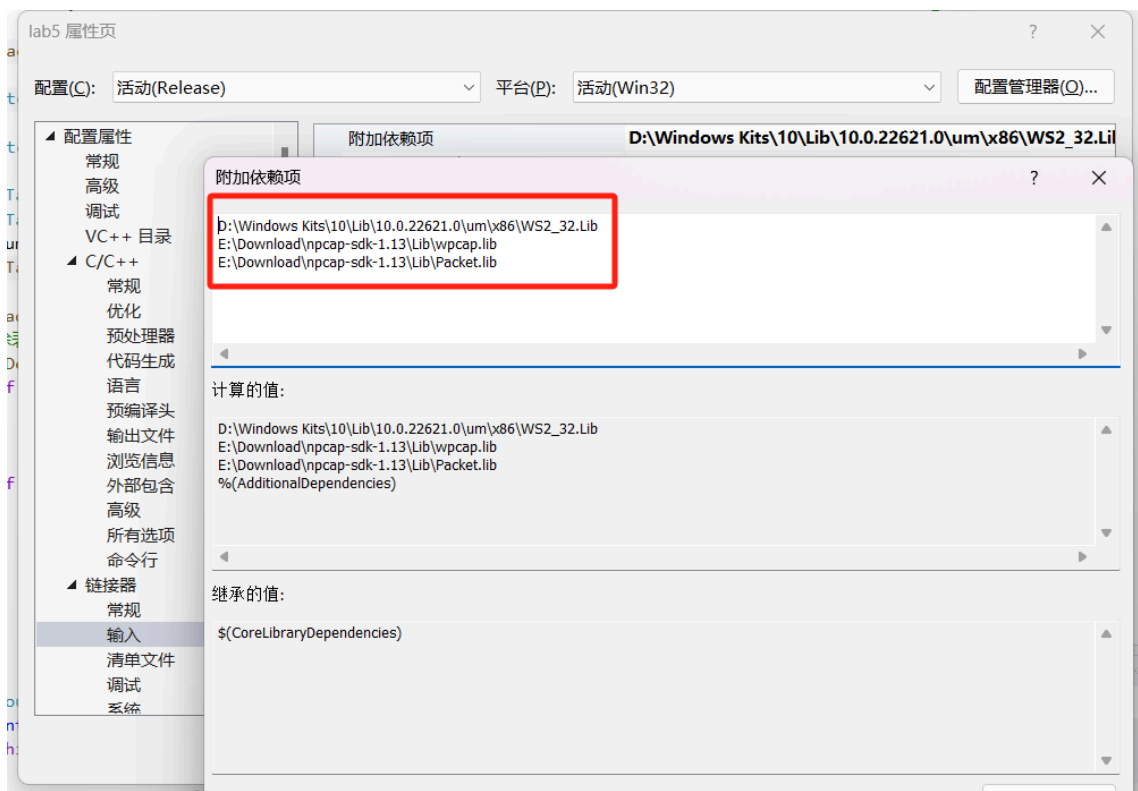


2. 附加库目录: ..\npcap-sdk-1.13\Lib



3. 附加依赖项:

```
D:\Windows Kits\10\Lib\10.0.22621.0\um\x86\ws2_32.Lib
E:\Download\npcap-sdk-1.13\Lib\wpcap.lib
E:\Download\npcap-sdk-1.13\Lib\Packet.lib
```

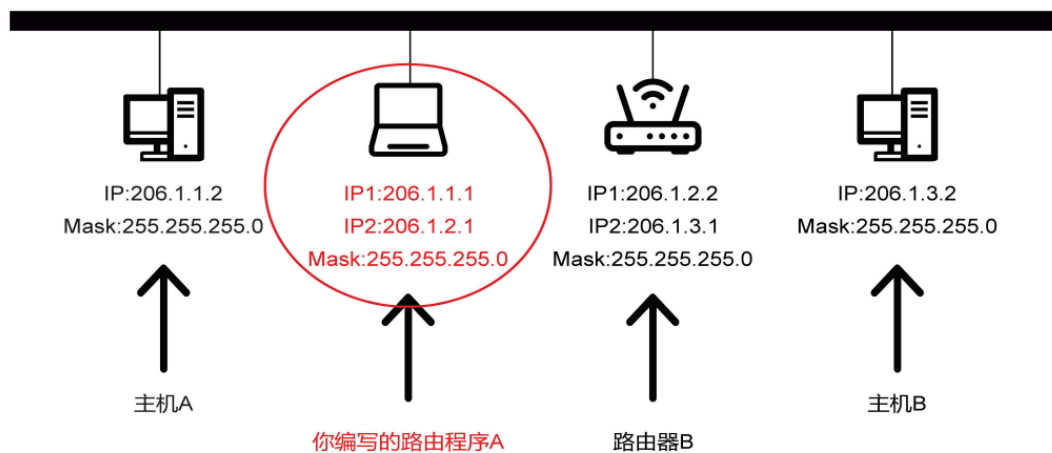


注意：本次实验不同于以往，前几次实验使用 x64 编译，链接器选择的都是 x64 版本，但是这次实验编译环境是在 Release x86 环境下，链接器均要换到 x86 版本。

实验设计

实验运行环境

实验环境拓扑图：



本次实验在四台虚拟机上运行，虚拟机 1 和 4 作为主机，2 和 3 作为路由器。其中 3 为配置好的路由器，我们需要在 2 上运行完成的路由程序。最后在主 1 上 ping 主机 4。

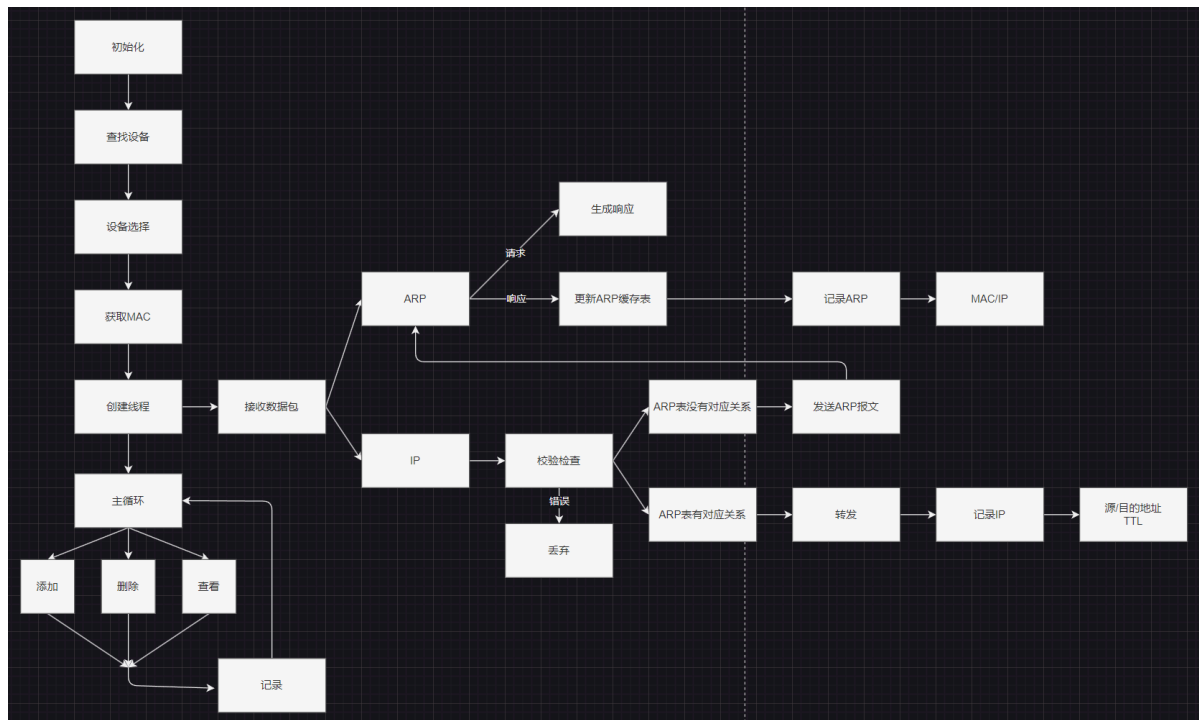
本次实验可以分为以下几部分：

1. 准备工作：

- 查询本机中的网卡，编号并输出网卡名称、描述、IP 地址即子网掩码
- 根据用户输入的网卡编号，打开网卡获取双 IP 地址

- 伪造 ARP 报文发送给自己，获取本机 MAC 地址
 - 输出选择界面，等待用户输入。
2. 接收消息并转发，输出记录：
- 接收消息：
 - 捕获数据包
 - 判断是 IP 数据包还是 ARP 数据报
 - 转发：
 - 修改 MAC 和 IP
 - 修改 TTL
 - 重新设置校验和
 - 记录：
 - 解析数据包头部，提取 IP 地址和 MAC 地址
 - 格式化输出
3. 用户操作：
- 用户输入：
 - 选择网卡
 - 选择操作：添加路由表项/删除路由表项/输出路由表

实验流程图：



相关代码

先定义实验中所需的数据结构体。

`Frame_Header`：以太网帧头部结构

`ARP_Frame`：ARP 数据报结构

`IP_Header`：IP 数据报头部结构

`Data`：完整的数据包结构，包含以太网帧头部和 IP 数据报头部

`Send_Packet`：发送数据包结构

```

struct Frame_Header {
    BYTE DesMAC[6];
    BYTE SrcMAC[6];
    WORD FrameType;
};

struct ARP_Frame {
    Frame_Header FrameHeader;
    WORD HardwareType;
    WORD ProtocolType;
    BYTE HLen;
    BYTE PLen;
    WORD op;
    BYTE SrcMAC[6];
    DWORD SrcIP;
    BYTE DesMAC[6];
    DWORD DesIP;
};

struct IP_Header {
    BYTE Version;
    BYTE TOS;
    WORD TotLen;
    WORD ID;
    WORD Flagoffset;
    BYTE TTL;
    BYTE Protocol;
    WORD Checksum;
    ULONG SrcIP;
    ULONG DstIP;
};

struct Data {
    Frame_Header FrameHeader;
    IP_Header IPHeader;
    char buf[0x80];
};

struct Send_Packet {
    BYTE PktData[2000];
    ULONG DestIP;
    bool flag = 1;
    clock_t time;
};

```

接下来定义一些实验中需要的类。

RouteTableEntry: 路由表项结构

RouteTable: 路由表结构

ARPTable: ARP 高速缓存表结构

```

class RouteTableEntry {
public:
    DWORD netmask;
    DWORD destnet;
    DWORD nextip;

```

```

int type;
RouteTableEntry* nextitem;

RouteTableEntry() { memset(this, 0, sizeof(*this)); };
RouteTableEntry(DWORD netmask, DWORD dstnet, int type, DWORD nextip = 0) {
    this->netmask = netmask;
    this->destnet = dstnet;
    this->nextip = nextip;
    this->type = type;
}

void print();
};

class RouteTable {
public:
    RouteTableEntry* head;
    RouteTableEntry* tail;
    int num;
    RouteTable() {
        num = 0;
        head = new RouteTableEntry(inet_addr(mymask[0]), (inet_addr(myip[0])) &
(inet_addr(mymask[0])), 0);
        tail = new RouteTableEntry;
        head->nextitem = tail;

        RouteTableEntry* temp = new RouteTableEntry;
        temp->destnet = (inet_addr(myip[1])) & (inet_addr(mymask[1]));;
        temp->netmask = inet_addr(mymask[1]);
        temp->type = 0;
        add(temp);

    }

    void add(RouteTableEntry* newt);
    void Delete(int index);
    void print();
    DWORD find(DWORD destip);
};

class ARPTable {
public:
    int arpnum = 0;
    DWORD IP;
    BYTE mac[6];

    void add(DWORD ip, BYTE mac[6]);
    int find(DWORD ip, BYTE mac[6]);
};

```

其次对类中所需操作进行定义。

RouteTableEntry

print() 函数用于输出路由表项的信息。输出子网掩码 MASK、目的网络 Tar-Net 和下一跳地址 Next-IP。

```

void RouteTableEntry::print() {
    in_addr addr;
    memcpy(&addr, &netmask, sizeof(netmask));
    printf("MASK: %s\n", inet_ntoa(addr));
    memcpy(&addr, &destnet, sizeof(destnet));
    printf("Tar-Net: %s\n", inet_ntoa(addr));
    memcpy(&addr, &nextip, sizeof(nextip));
    printf("Next-IP: %s\n", inet_ntoa(addr));
}

```

RouteTable

add() 函数用于向路由表中添加新的路由表项。如果新的路由表项是直接连接的，则将其插入到路由表的头部。否则，将其插入到合适的位置，使得路由表项按照子网掩码从大到小排序。

Delete() 函数用于从路由表中删除指定位置的路由表项。如果指定位置超出了路由表的范围，则输出错误信息。

print() 函数用于输出路由表的信息。首先输出路由表的总项数，然后遍历路由表，依次输出每一项的信息。

find() 函数用于在路由表中查找指定目的 IP 地址对应的下一跳 IP 地址。首先遍历路由表，依次比较每一项的目的网络和子网掩码。如果找到了匹配的路由表项，则返回下一跳 IP 地址。如果没有找到匹配的路由表项，则输出错误信息并返回 -1。

```

void RouteTable::add(RouteTableEntry* newt) {
    num++;
    if (newt->type == 0) {
        newt->nextitem = head->nextitem;
        head->nextitem = newt;
        return;
    }

    RouteTableEntry* cur = head;
    while (cur->nextitem != tail) {
        if (cur->nextitem->type != 0 && cur->nextitem->netmask <= newt->netmask)
        {
            break;
        }
        cur = cur->nextitem;
    }

    newt->nextitem = cur->nextitem;
    cur->nextitem = newt;
}

void RouteTable::Delete(int index) {
    if (index > num) {
        printf("Routing-Table Entries %d beyond the scope.\n", index);
        return;
    }
    if (index == 0) {
        if (head->type == 0) {
            printf("Delete failed!\n");
        }
        else {
            head = head->nextitem;
        }
    }
}

```

```

        return;
    }
    RouteTableEntry* cur = head;
    int i = 0;
    while (i < index - 1 && cur->nextitem != tail) {
        i++;
        cur = cur->nextitem;
    }
    if (cur->nextitem->type == 0) {
        printf("Delete failed!\n");
    }
    else {
        cur->nextitem = cur->nextitem->nextitem;
    }
}

void RouteTable::print() {
    RouteTableEntry* cur = head;
    int i = 1;
    while (cur != tail) {
        printf("%d Routing-Table Entry\n", i);
        cur->print();
        cur = cur->nextitem;
        i++;
    }
}

DWORD RouteTable::find(DWORD destip) {
    DWORD result;
    RouteTableEntry* cur = head;
    while (cur != tail) {
        result = destip & cur->netmask;
        if (result == cur->destnet) {
            if (cur->type == 1) {
                return cur->nextip;
            }
            else if (cur->type == 0) {
                return destip;
            }
        }
        cur = cur->nextitem;
    }
    printf("No Routing-Table Entry.\n");
    return -1;
}

```

ARPTable

`add()` 函数用于向 ARP 高速缓存表中添加新的 ARP 表项。将指定的 IP 地址和 MAC 地址存储到 ARP 高速缓存表中。

`find()` 函数用于在 ARP 高速缓存表中查找指定 IP 地址对应的 MAC 地址。如果找到了匹配的 ARP 表项，则将其 MAC 地址复制到指定的缓冲区中，并返回 1。如果没有找到匹配的 ARP 表项，则返回 0。

```

void ARPTable::add(DWORD ip, BYTE mac[6]) {
    extern ARPTable arp_table[50];
    arp_table[arpnum].IP = ip;
}

```

```

    for (int i = 0; i < 6; i++) {
        arp_table[arpnum].mac[i] = mac[i];
    }

    arpnum++;
}
int ARPTable::find(DWORD ip, BYTE mac[6]) {
    extern ARPTable arp_table[50];
    for (int i = 0; i < arpnum; i++) {
        if (ip == arp_table[i].IP) {
            for (int j = 0; j < 6; j++) {
                mac[j] = arp_table[i].mac[j];
            }
            return 1;
        }
    }
    return 0;
}

```

`SendPacket()` 函数用于发送数据包。首先将数据包的源 MAC 地址和目的 MAC 地址进行交换，然后将 TTL 减 1。如果 TTL 小于 0，则直接返回。最后调用 `SetChecksum()` 函数计算校验和，并使用 `pcap_sendpacket()` 函数发送数据包。

```

void SendPacket(Data data, BYTE nextmac[]) {
    Data* temp = (Data*)&data;
    memcpy(temp->FrameHeader.SrcMAC, temp->FrameHeader.DesMAC, 6);
    memcpy(temp->FrameHeader.DesMAC, nextmac, 6);
    temp->IPHeader.TTL -= 1;
    if (temp->IPHeader.TTL < 0)
        return;
    SetChecksum(temp);

    printf("Host-MAC Add: \t ");
    for (int i = 0; i < 5; i++) {
        printf("%02X-", temp->FrameHeader.SrcMAC[i]);
    }
    printf("%02X\n", temp->FrameHeader.SrcMAC[5]);

    printf("Tar-MAC Add: \t");
    for (int i = 0; i < 5; i++) {
        printf("%02X-", temp->FrameHeader.DesMAC[i]);
    }
    printf("%02X\n", temp->FrameHeader.DesMAC[5]);
    printf("Host-IP Add: \t ");
    in_addr addr;
    memcpy(&addr, &temp->IPHeader.SrcIP, sizeof(temp->IPHeader.SrcIP));
    printf("%s ", inet_ntoa(addr));
    printf("\n");
    printf("Tar-IP Add:\t ");
    memcpy(&addr, &temp->IPHeader.DstIP, sizeof(temp->IPHeader.DstIP));
    printf("%s ", inet_ntoa(addr));
    printf("\n");
    printf("TTL: %d\n", temp->IPHeader.TTL);
    pcap_sendpacket(point, (const u_char*)temp, 74);
}

```

```
}
```

`receive` 是线程处理函数。这是路由器程序的核心函数之一，负责处理所有接收到的数据包。

首先，它创建了一个 `ARPTable` 对象 `arptable`，用于存储 ARP 表项。然后，它进入一个无限循环，不断接收数据包。

接下来是 ARP 报文的处理过程。如果接收到的数据包是一个 ARP 响应包，会进行更新 ARP 高速缓存表，同时检查缓存中是否有可以发送的数据包。同时会进行超时检查。

如果接收到的是 IP 数据包，首先会对 MAC 地址进行检验，在对 IP 进行校验和检验。通过后便检查是否为发送给自己的包，不是的话进行查表，然后进行转发。

数据包转发通过查表后分为直接投递和下一跳转发。如果 ARP 表中没有目标 MAC，则会先将数据包存入缓冲区，有对应关系便直接转发。

```
DWORD WINAPI receive(LPVOID lparam) {
    ARPTable arptable;
    RouteTable rtable = *(RouteTable*)(LPVOID)lparam;
    while (1) {
        pcap_pkthdr* pkt_header;
        const u_char* packetData;

        while (1) {
            int result = pcap_next_ex(point, &pkt_header, &packetData);
            if (result)
                break;
        }

        Frame_Header* header = (Frame_Header*)packetData;

        if (ntohs(header->FrameType) == 0x806) {
            ARP_Frame* data = (ARP_Frame*)packetData;

            printf("Host-MAC Add: \t ");
            for (int i = 0; i < 5; i++) {
                printf("%02X-", data->FrameHeader.SrcMAC[i]);
            }
            printf("%02X\n", data->FrameHeader.SrcMAC[5]);

            printf("Tar-MAC Add: \t");
            for (int i = 0; i < 5; i++) {
                printf("%02X-", data->FrameHeader.DesMAC[i]);
            }
            printf("%02X\n", data->FrameHeader.DesMAC[5]);
            printf("Host-IP Add: \t ");
            in_addr addr;
            memcpy(&addr, &data->SrcIP, sizeof(data->SrcIP));
            printf("%s ", inet_ntoa(addr));
            printf("\n");
            printf("Tar-IP Add: \t ");
            memcpy(&addr, &data->DesIP, sizeof(data->DesIP));
            printf("%s ", inet_ntoa(addr));
            printf("\n");

            if (data->op == ntohs(0x0002)) {
                BYTE tempmac[6];
                if (arptable.find(data->SrcIP, tempmac));
            }
        }
    }
}
```

```

        else
            arptable.add(data->SrcIP, data->SrcMAC);

    for (int i = 0; i < bufsize; i++)
    {
        if (Buffer[i].flag == 0)
            continue;
        if (clock() - Buffer[i].time >= 6000) {
            Buffer[i].flag = 0;
            continue;
        }
        if (Buffer[i].DestIP == data->SrcIP) {
            Data* data_send = (Data*)Buffer[i].PktData;
            Data temp = *data_send;
            SendPacket(temp, data->SrcMAC);
            Buffer[i].flag = 0;
            break;
        }
    }
}

}

if (Compare(header->DesMAC, mymac) && ntohs(header->FrameType) == 0x800)
{
    Data* data = (Data*)packetData;
    if (!Check(data)) {
        printf("Checksum Error.\n");
        continue;
    }

    printf("Host-MAC Add: \t ");
    for (int i = 6; i < 12; ++i) {
        printf("%02X", packetData[i]);
        if (i < 11) printf("-");
    }
    printf("\n");
    printf("Tar-MAC Add:\t");
    for (int i = 0; i < 6; ++i) {
        printf("%02X", packetData[i]);
        if (i < 5) printf("-");
    }
    printf("\n");
    printf("Host-IP Add: \t ");
    for (int i = 26; i < 30; ++i) {
        printf("%d", packetData[i]);
        if (i < 29) printf(".");
    }
    printf("\n");
    printf("Tar-IP Add: \t ");
    for (int i = 30; i < 34; ++i) {
        printf("%d", packetData[i]);
        if (i < 33) printf(".");
    }
    printf("\n");
    printf("TTL: %d\n", data->IPHeader.TTL);
}

```

```

if (data->IPHeader.DstIP == inet_addr(myip[0])
    || data->IPHeader.DstIP == inet_addr(myip[1])) {
    printf("Selfcheck\n");
    continue;
}
DWORD destip = data->IPHeader.DstIP;
DWORD nextdestip = rtable.find(destip);

if (nextdestip == -1) {
    printf("Routing-Table Entries! \n");
    continue;
}
else {
    in_addr next;
    next.s_addr = nextdestip;
    printf("Next-IP: %s\n", inet_ntoa(next));

    Data* temp2 = (Data*)packetData;
    Data temp = *temp2;
    BYTE mac[6];

    if (nextdestip == destip) {
        if (!arptable.find(destip, mac)) {
            int flag2 = 0;
            for (int i = 0; i < bufsize; i++) {
                if (Buffer[i].flag == 0) {
                    flag2 = 1;
                    memcpy(Buffer[i].PktData, packetData, pkt_header-
>len);

                    Buffer[i].flag = 1;
                    Buffer[i].time = clock();
                    Buffer[i].DestIP = destip;
                    GetTarMac(destip, mac);
                    break;
                }
            }

            if (flag2 == 0 && bufsize < 50) {
                memcpy(Buffer[bufsize].PktData, packetData,
pkt_header->len);

                Buffer[bufsize].flag = 1;
                Buffer[bufsize].time = clock();
                Buffer[bufsize].DestIP = destip;
                bufsize++;
                GetTarMac(destip, mac);
            }
            else
                printf("Buffer full.\n");
        }
        else if (arptable.find(destip, mac))
            SendPacket(temp, mac);
    }
    else {
        if (!arptable.find(nextdestip, mac)) {
            int flag3 = 0;

```


检查 3 号设备的路由表项是否添加成功：

```
C:\WINDOWS\system32\cmd.exe
C:\Documents and Settings\Administrator>route print

IPv4 Route Table
=====
Interface List
0x1 ..... MS TCP Loopback interface
0x10003 ...00 0c 29 d4 a6 fd ..... Intel(R) PRO/1000 MT Network Connection
=====
Active Routes:
Network Destination        Netmask          Gateway          Interface        Metric
206.1.1.0                  255.255.255.0    206.1.2.1        206.1.2.2        1
206.1.2.0                  255.255.255.0    206.1.2.2        206.1.2.2        10
206.1.2.2                  255.255.255.255  127.0.0.1        127.0.0.1        10
206.1.2.255               255.255.255.255  206.1.2.2        206.1.2.2        10
206.1.3.0                  255.255.255.0    206.1.3.1        206.1.2.2        10
206.1.3.1                  255.255.255.255  127.0.0.1        127.0.0.1        10
206.1.3.255               255.255.255.255  206.1.3.1        206.1.2.2        10
224.0.0.0                  240.0.0.0        206.1.2.2        206.1.2.2        10
255.255.255.255           255.255.255.255  206.1.2.2        206.1.2.2        1
=====
Persistent Routes:
None

C:\Documents and Settings\Administrator>a
```

将生成的路由器程序拖入设备 2 中，运行：

```
C:\Documents and Settings\Administrator\桌面\lab5.exe
1. \Device\NPF_{3AC00148-1BFF-47AF-8441-F1D14A619031}\{Intel(R) PRO/1000 MT Network Connection}
IP: 206.1.2.1
Mask: 255.255.255.0
IP: 206.1.1.1
Mask: 255.255.255.0
Select a device (1-1): 1
<\Device\NPF_{3AC00148-1BFF-47AF-8441-F1D14A619031}\{Intel(R) PRO/1000 MT Network Connection}
IP: 206.1.2.1
MASK: 255.255.255.0
<\Device\NPF_{3AC00148-1BFF-47AF-8441-F1D14A619031}\{Intel(R) PRO/1000 MT Network Connection}
IP: 206.1.2.1
MASK: 255.255.255.0
MAC: 00-0C-29-21-54-9B
1 Routing-Table Entry
MASK: 255.255.255.0
Tar-Net: 206.1.2.0
Next-IP: 0.0.0.0
2 Routing-Table Entry
MASK: 255.255.255.0
Tar-Net: 206.1.1.0
Next-IP: 0.0.0.0
1.Add Routing-Table Entries
2.Delete Routing-Table Entries
3.Print Routing-Table
```

程序开始先列举设备中的网卡，输入对应的网卡后显示相关信息，然后出现现在的路由表。接下来是用户的循环。

```

MAC:      00-0C-29-21-54-9B
1 Routing-Table Entry
MASK: 255.255.255.0
Tar-Net: 206.1.2.0
Next-IP: 0.0.0.0
2 Routing-Table Entry
MASK: 255.255.255.0
Tar-Net: 206.1.1.0
Next-IP: 0.0.0.0
1.Add Routing-Table Entries
2.Delete Routing-Table Entries
3.Print Routing-Table
1
Mask: 255.255.255.0
Tar-Net: 206.1.3.0
Next-IP: 206.1.2.2
1.Add Routing-Table Entries
2.Delete Routing-Table Entries
3.Print Routing-Table

```

用户输入 1，添加路由表项。子网掩码 255.255.255.0，目的网络 206.1.3.0，下一跳地址 206.1.2.2。

```

1.Add Routing-Table Entries
2.Delete Routing-Table Entries
3.Print Routing-Table
3
1 Routing-Table Entry
MASK: 255.255.255.0
Tar-Net: 206.1.2.0
Next-IP: 0.0.0.0
2 Routing-Table Entry
MASK: 255.255.255.0
Tar-Net: 206.1.1.0
Next-IP: 0.0.0.0
3 Routing-Table Entry
MASK: 255.255.255.0
Tar-Net: 206.1.3.0
Next-IP: 206.1.2.2
1.Add Routing-Table Entries
2.Delete Routing-Table Entries
3.Print Routing-Table

```

添加完后输入 3，查看路由表，可看到序号为 3 的路由表项为刚才添加的。

Ethernet adapter 本地连接:

```

Connection-specific DNS Suffix  . :
IP Address. . . . . : 206.1.1.2
Subnet Mask . . . . . : 255.255.255.0
Default Gateway . . . . . : 206.1.1.1

```

C:\Documents and Settings\Administrator>ping 206.1.3.2

Pinging 206.1.3.2 with 32 bytes of data:

```

Reply from 206.1.3.2: bytes=32 time=2058ms TTL=126
Reply from 206.1.3.2: bytes=32 time=1188ms TTL=126
Reply from 206.1.3.2: bytes=32 time=1142ms TTL=126
Reply from 206.1.3.2: bytes=32 time=1190ms TTL=126

```

Ping statistics for 206.1.3.2:

```

    Packets: Sent = 4, Received = 4, Lost = 0 (0% loss),
Approximate round trip times in milli-seconds:
    Minimum = 1142ms, Maximum = 2058ms, Average = 1394ms

```

此时在主机 1 上 ping 主机 4，发现可以 ping 通。

```

1. Add Routing-Table Entries
2. Delete Routing-Table Entries
3. Print Routing-Table
3
1 Routing-Table Entry
MASK: 255.255.255.0
Tar-Net: 206.1.2.0
Next-IP: 0.0.0.0
2 Routing-Table Entry
MASK: 255.255.255.0
Tar-Net: 206.1.1.0
Next-IP: 0.0.0.0
1. Add Routing-Table Entries
2. Delete Routing-Table Entries
3. Print Routing-Table

```

此时回到主机 2 上，输入操作 2，删除路由表项 3。再查看路由表，发现可以删除。

```

1. Add Routing-Table Entries
2. Delete Routing-Table Entries
3. Print Routing-Table
3
1 Routing-Table Entry
MASK: 255.255.255.0
Tar-Net: 206.1.2.0
Next-IP: 0.0.0.0
2 Routing-Table Entry
MASK: 255.255.255.0
Tar-Net: 206.1.1.0
Next-IP: 0.0.0.0
1. Add Routing-Table Entries
2. Delete Routing-Table Entries
3. Print Routing-Table
2
Delete Num: 1
Delete failed!
1. Add Routing-Table Entries
2. Delete Routing-Table Entries
3. Print Routing-Table
3
1 Routing-Table Entry
MASK: 255.255.255.0
Tar-Net: 206.1.2.0
Next-IP: 0.0.0.0
2 Routing-Table Entry
MASK: 255.255.255.0
Tar-Net: 206.1.1.0
Next-IP: 0.0.0.0
1. Add Routing-Table Entries
2. Delete Routing-Table Entries
3. Print Routing-Table

```

尝试删除默认路由表项，提示 `Delete failed!`，再查看路由表发现没有删除。

日志记录:

```
C:\Documents and Settings\Administrator\桌面\lab5.exe
Host-MAC Add: 00-0C-29-21-54-9B
Tar-MAC Add: 00-0C-29-DB-A5-3F
Host-IP Add: 206.1.3.2
Tar-IP Add: 206.1.1.2
TTL: 126
Host-MAC Add: 00-0C-29-DB-A5-3F
Tar-MAC Add: 00-0C-29-21-54-9B
Host-IP Add: 206.1.1.2
Tar-IP Add: 206.1.3.2
TTL: 128
Next-IP: 206.1.2.2
Host-MAC Add: 00-0C-29-21-54-9B
Tar-MAC Add: 00-0C-29-D4-A6-FD
Host-IP Add: 206.1.1.2
Tar-IP Add: 206.1.3.2
TTL: 127
Host-MAC Add: 00-0C-29-D4-A6-FD
Tar-MAC Add: 00-0C-29-21-54-9B
Host-IP Add: 206.1.3.2
Tar-IP Add: 206.1.1.2
TTL: 127
Next-IP: 206.1.1.2
Host-MAC Add: 00-0C-29-21-54-9B
Tar-MAC Add: 00-0C-29-DB-A5-3F
Host-IP Add: 206.1.3.2
Tar-IP Add: 206.1.1.2
TTL: 126
Host-MAC Add: 00-0C-29-DB-A5-3F
Tar-MAC Add: 00-0C-29-21-54-9B
Host-IP Add: 206.1.1.2
Tar-IP Add: 206.1.3.2
TTL: 128
Next-IP: 206.1.2.2
Host-MAC Add: 00-0C-29-21-54-9B
Tar-MAC Add: 00-0C-29-D4-A6-FD
Host-IP Add: 206.1.1.2
Tar-IP Add: 206.1.3.2
TTL: 127
Host-MAC Add: 00-0C-29-D4-A6-FD
Tar-MAC Add: 00-0C-29-21-54-9B
Host-IP Add: 206.1.3.2
Tar-IP Add: 206.1.1.2
TTL: 127
Next-IP: 206.1.1.2
Host-MAC Add: 00-0C-29-21-54-9B
Tar-MAC Add: 00-0C-29-DB-A5-3F
Host-IP Add: 206.1.3.2
Tar-IP Add: 206.1.1.2
TTL: 126
```